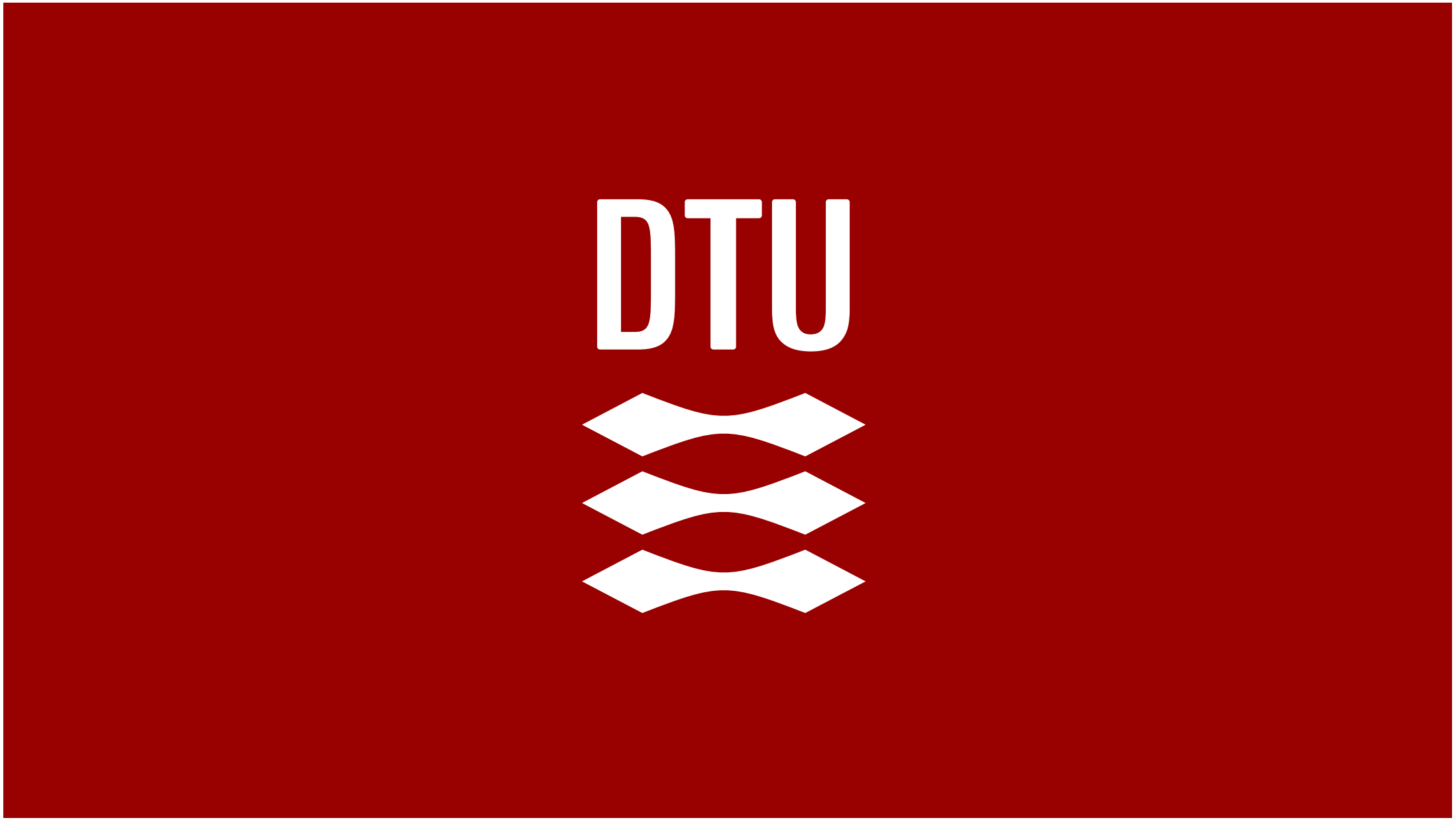
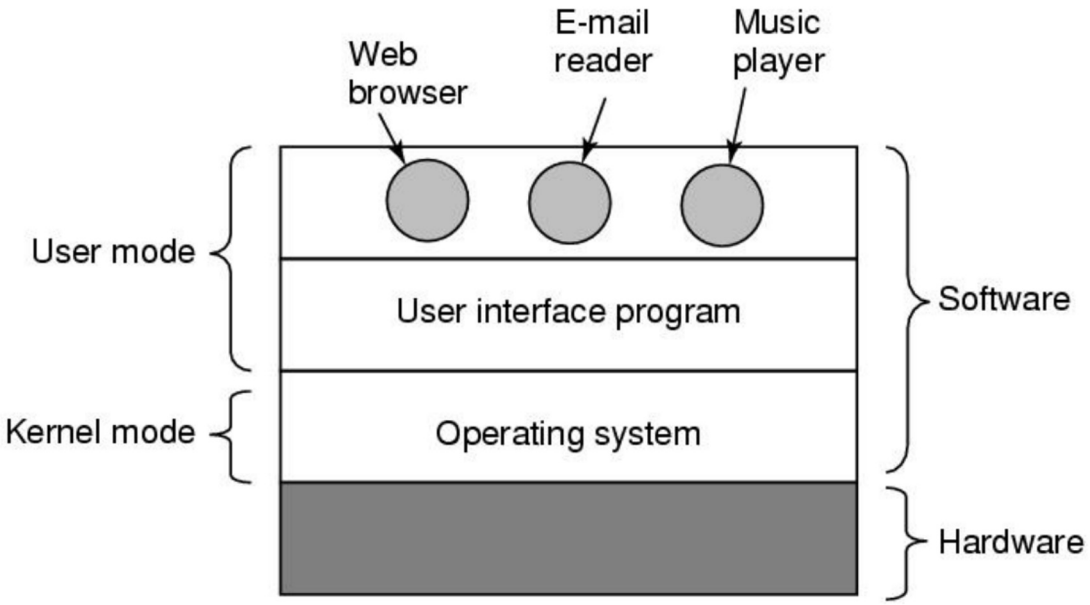

DTU

02159 – Operating Systems

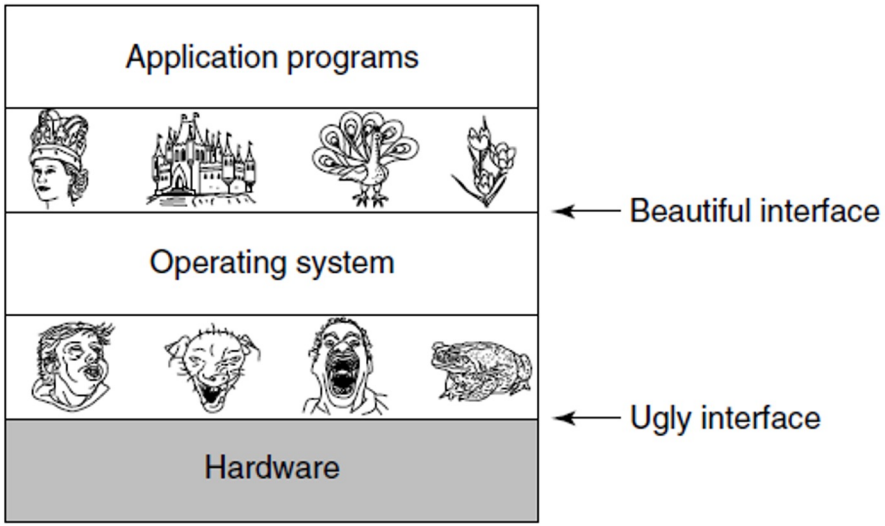
# Week 2: Computer Organisation and OS Structures

**Xenofon (Fontas) Fafoutis**
Professor
xefa@dtu.dk
www.compute.dtu.dk/~xefa

## Operating Systems abstract and manage resources



# From nasty hardware to beautiful abstraction

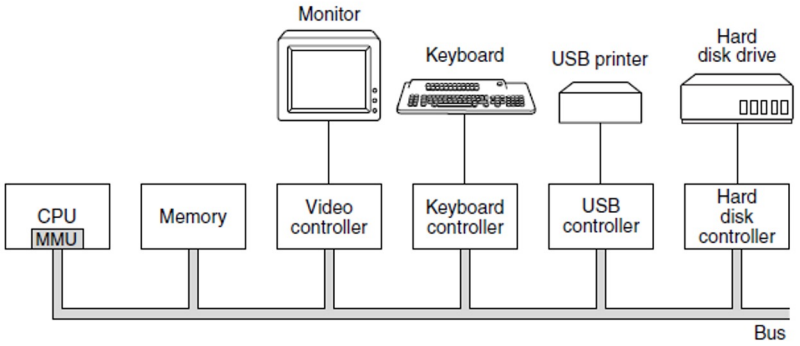


# Computer Hardware Review

- The Operating System talks directly with the hardware
- What are the main parts of a computer system?

# Components of a Modern Computer

- One or more processors
- Main memory
- Disks
- CD/DVD drives
- Printers
- Keyboard
- Mouse
- Display
- Network interfaces
- I/O devices
- ...



# Processor (CPU)

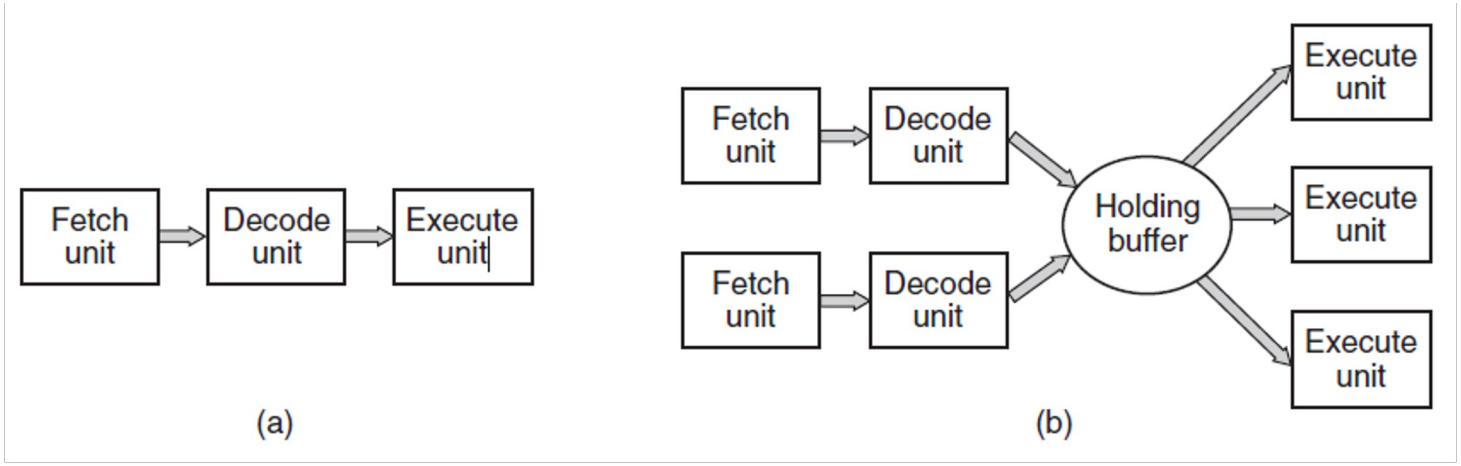
- The **brain** of a computer
- What does it do?

# Processor (CPU)

- The **brain** of a computer
- What does it do?
  - It fetches instructions from the memory and **executes** them
    - Data handling instructions (read, write, etc)
    - Arithmetic and Logic instructions (add, divide, compare, etc)
    - Control flow instructions (jump, call, etc)
- Registers
  - General purpose registers (key variables, temp results, etc)
  - Special registers (program counter, stack pointer, etc)

# Introducing some parallelism

- Instruction **Pipelining**: Separate fetch, decode, and execute units
- **Superscalar** CPU
  - Specialised execution units (e.g. a floating-point unit)



# Multithreaded and Multicore Processors

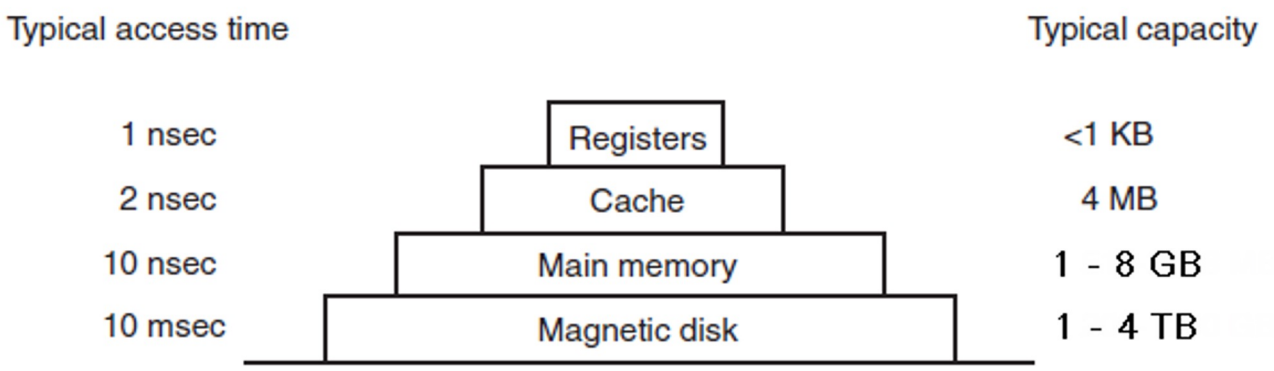
- Multithreading
  - Pseudo-parallel execution of two threads
  - CPU is able to hold the state of two execution threads and switch between them super fast
  - Example: One thread is fetching data from memory (slow), CPU switches to other thread while the first is waiting

- Multiple Processors (4, 8, 16, …)
  - True parallelism
  - Graphics Processing Unit (GPU)
    - Extreme parallelism

# Memory

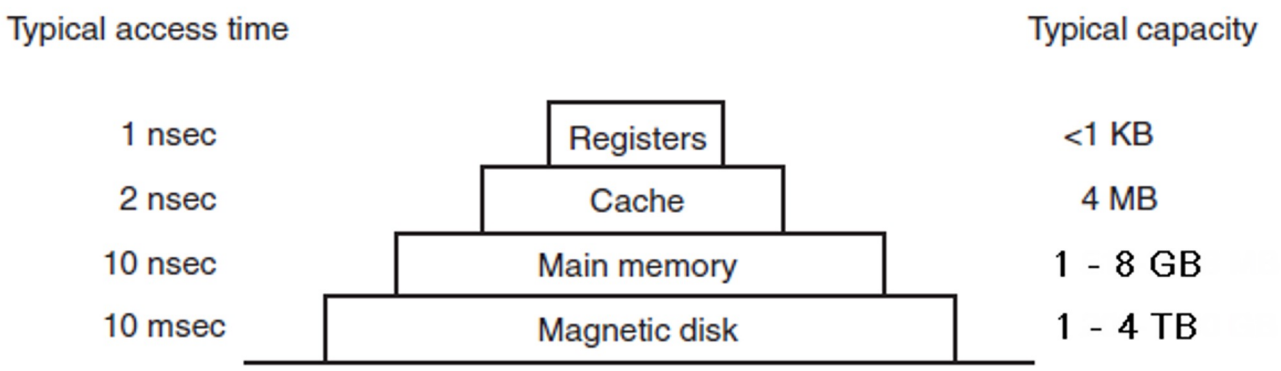
- We want memory to be **fast**, **large**, and **cheap**.
  - We can only pick two out of three!

- **Layered** memory



# Layered Memory

- In a layered memory architecture, we want to keep data that is used frequently in the **fast memory**
- Question: How do I know which data to keep in cache?





# How do I know which data to keep in cache?

- We don’t know… we have to guess!

- However, we can try to make good guesses…

# The Principle of Locality

*A programme accesses a relatively small portion of the address space at any instant of time*

Two different types of locality:

- **Temporal** locality (Locality in time): If an item is referenced, it will tend to be referenced again soon.
- **Spatial** locality (Locality in space): If an item is referenced, items whose addresses are close, tend to be referenced soon.

# Cache Memory: Implications for Programming

Example

```
int a[1000][1000];
int c = 0;

for (i=0; i<1000; i++)
        for (j=0; j<1000; j++)
                c += a[i][j];
```

- Which one is faster?

# Cache Memory: Implications for Programming

Example

```
int a[1000][1000];
int c = 0;

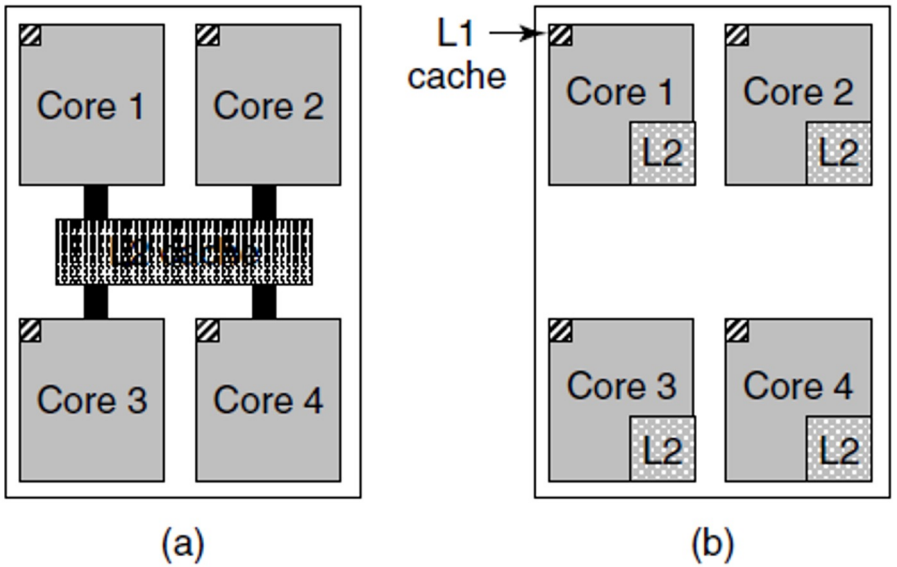
for (i=0; i<1000; i++)
        for (j=0; j<1000; j++)
                c += a[j][i];
```

- Which one is faster?

# Cache Memory: Implications for Programming

- Caches depend on locality to work. Your code must exhibit locality!
- Temporal locality
  - Make sure any data reuse is shortly after prior use
- Spatial locality
  - Cluster reused data together
  - Cluster together data accessed shortly after each other

# Some Quad-Core Architectures



# What is your cache structure?

```
> lscpu

> getconf -a | grep -i cache
```

# Input / Output Devices

- I/O devices generally have a **controller**
- A controller is a mini computer chip that physically controls the device on behalf of the computer
  - Controllers have processors (often non-programmable though)
  - Controllers have registers
  - Controllers may have memory
  - A controller **accepts commands** from the Operating System

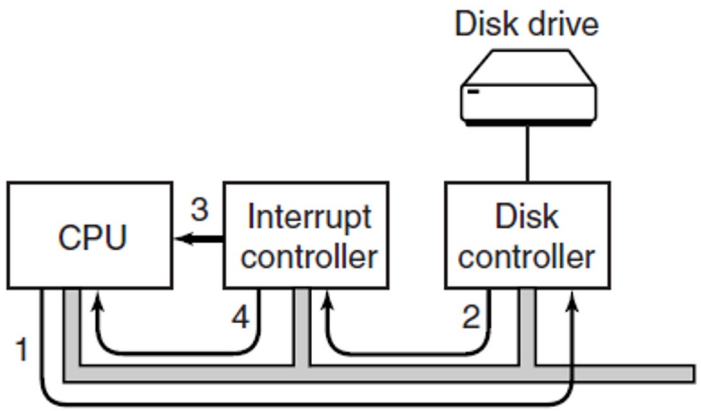
- **Device Driver**
  - A piece of software in the OS that talks to a controller
  - How do we put a driver in the kernel?
    - In the old days, we had to relink the kernel
    - Then, operating system loads drivers when it boots
    - Lately, drivers are loaded on the fly (plug and play)

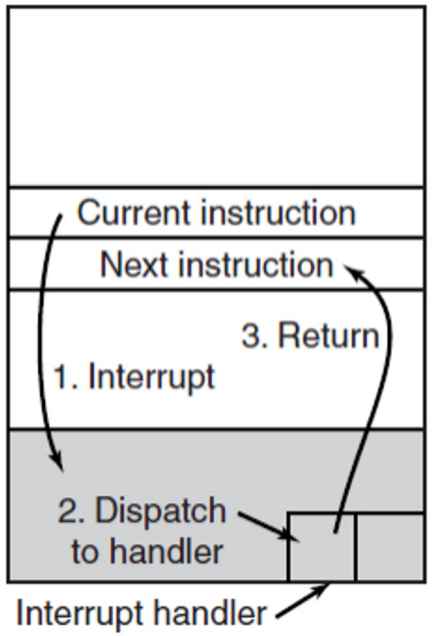
# Accessing the memory/registers of I/O devices

- Abstraction
  - Mapping them to the system's **address space**
  - Can read/write them as any other memory address

- Communication via special in/out instructions
  - Busy Waiting
    - Send command, check if done in loop
  - Interrupt-based
    - Send command and sleep; controller generates an **interrupt** to signal completion
  - Direct Memory Access (DMA)
    - A special chip that can move data between controllers and main memory without the involvement of the main CPU

# Interrupts


Disk drive
CPU
3
Interrupt controller
Disk controller
4
2
1


Current instruction
Next instruction
3. Return
1. Interrupt
2. Dispatch to handler
Interrupt handler

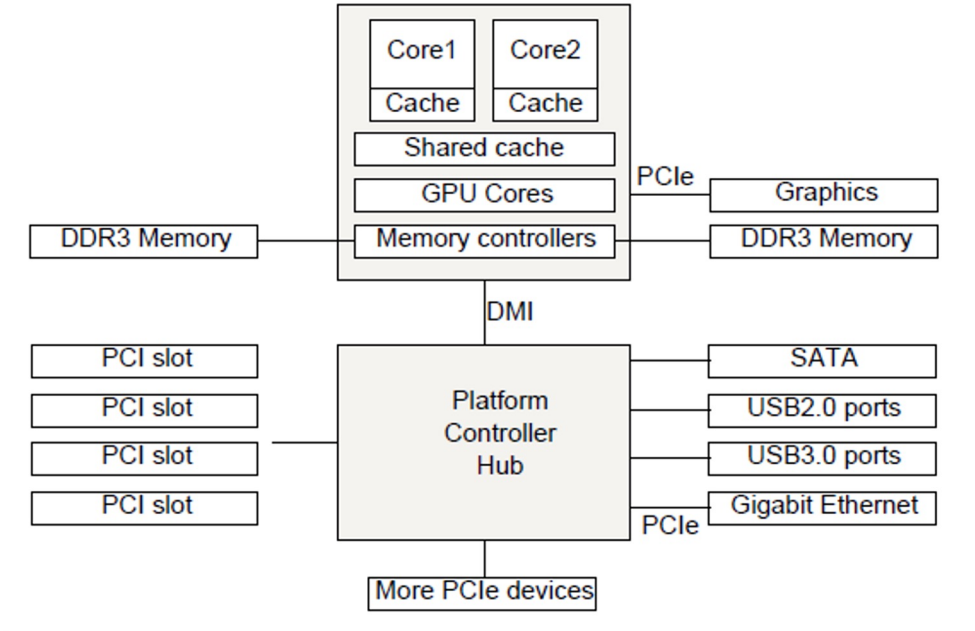
# Buses

- Communication lines between parts of the computer

- Serial Buses
  - One bit at a time
- Parallel
  - One word (e.g. 32-bits) at a time

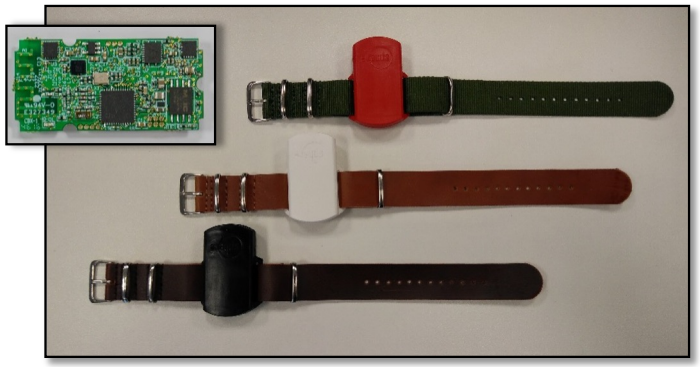
- Dedicated
  - Allocated exclusively to two components
- Shared
  - Multiple devices share it

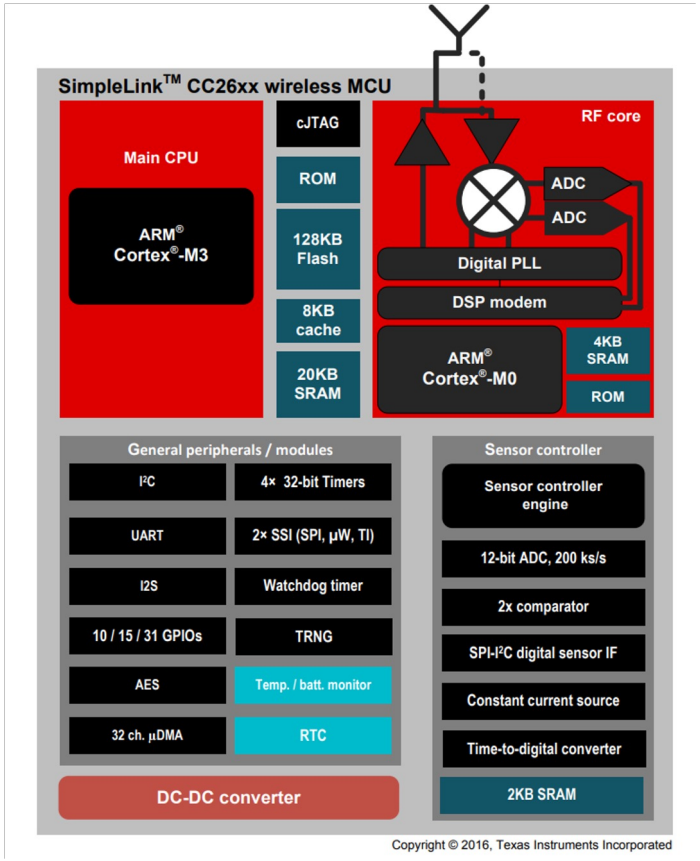
# The Structure of a x86 System



# The Structure of a Sensor System

- The SPHERE Wearable Sensor
  - CC2650 System-on-Chip
  - ADXL362 Accelerometer





# CPU Modes of Operation

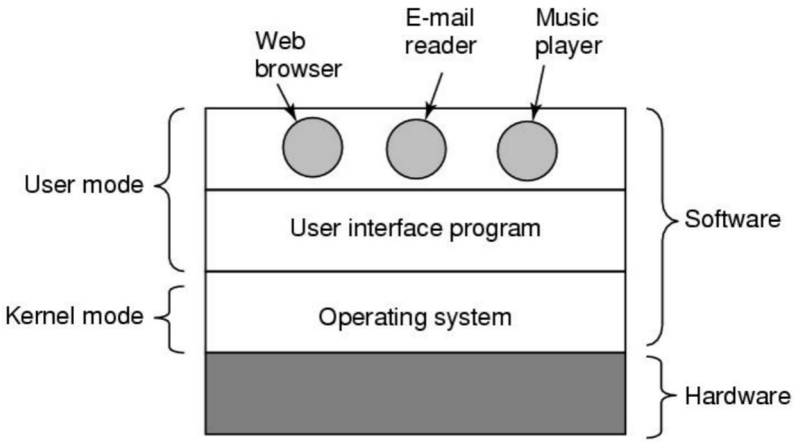
- Kernel mode
  - CPU can execute **every instruction** and use **all hardware**
  - The Operating System runs in kernel mode (typically)

- User mode
  - Allowed instructions and hardware resources are **limited**
  - All user programmes run in user mode
  - To access these protected resources the programme must make a **system call** (normal execution is suspended and operating system takes control, i.e. **trap**)

# Modes of Operation

- Kernel mode
  - **All** instructions and all hardware resources are accessible
- User mode
  - Accessible instructions and hardware resources are **limited**

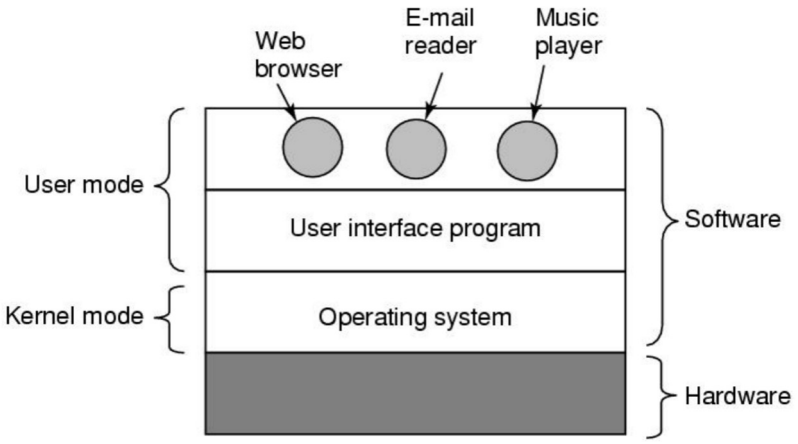
- System call
  - To access these protected resources the programme must make a **system call**



# Modes of Operation

- Kernel mode
  - **All** instructions and all hardware resources are accessible
- User mode
  - Accessible instructions and hardware resources are **limited**

- System call
  - To access these protected resources the programme must make a **system call**




**Why?**

# Why User Mode?
# Why not run everything in kernel mode?

- Enables the Operating System to **enforce** management policies
  - Applications can only access protected resources through the OS
- **Stability** through isolation
  - A bug in a user mode cannot take down the entire system
- **Security** in multi-user systems
- However, user mode is **costly**
  - Transitioning between kernel/user modes is expensive

# Operating System Structure

- An Operating System provides a lot of services
- Which of these operating system services should we run in **kernel mode** and which in **user mode**?

# Operating System Structure

- An Operating System provides a lot of services
- Which of these operating system services should we run in **kernel mode** and which in **user mode**?

***Stability vs Efficiency Trade-off***

# Monolithic Systems

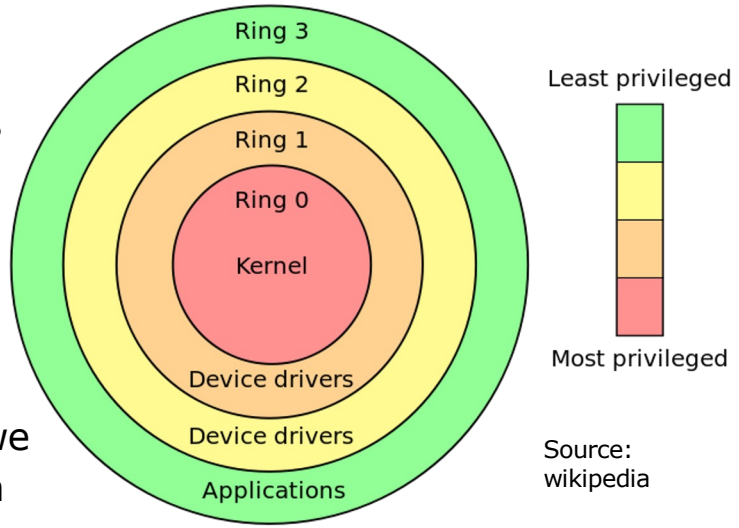
- The **entire OS** runs as a single programme in **kernel mode**
  - User programmes access OS services via system calls

- Advantages
  - Very efficient
    - Within the OS all resources/services are directly accessible

- Disadvantages
  - A bug in one OS service can take down the entire system
    - E.g. a buggy audio driver may accidentally write on the disk
  - Less flexible
    - Any system extensions require rebuilding the kernel

# Layered Systems

- Organised as a **hierarchy of layers**
- Inner layers are more privileged than outer layers




Source: wikipedia

- Advantages
  - High flexibility
    - Easy to extend as long as we adhere to the interface with layer above and below

- Disadvantages
  - Low efficiency
    - A procedure in an outer layer must make the equivalent of multiple system calls to access inner resources
  - Defining layers is not straightforward

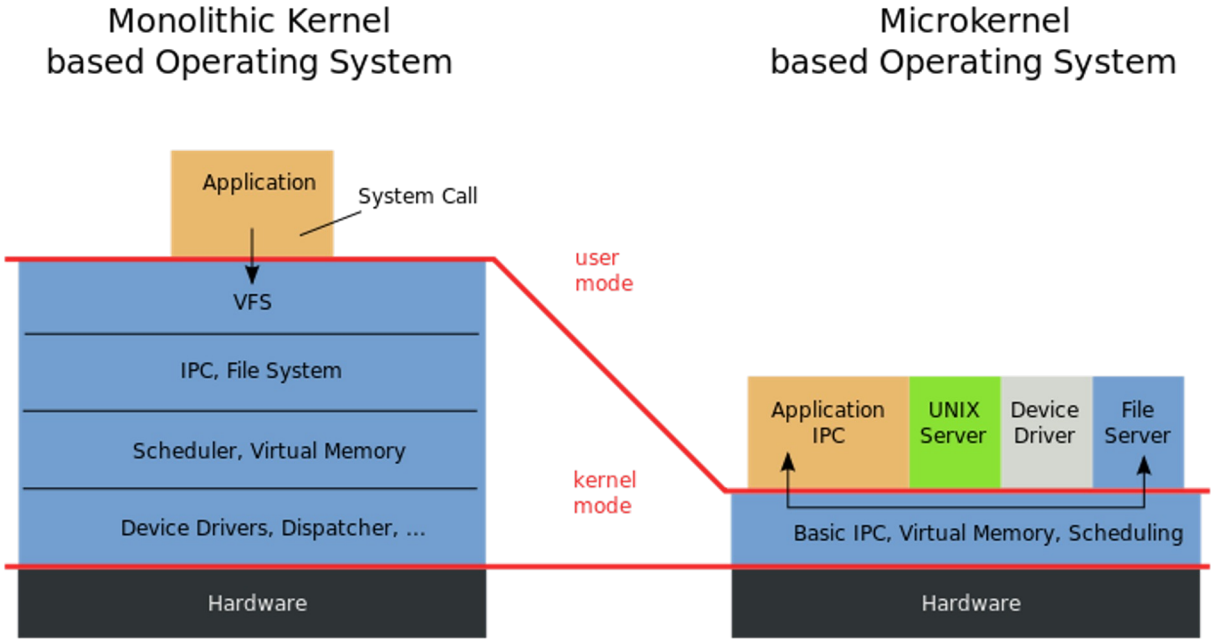
# Microkernels

- Keep kernel very **small**
  - Basic process/memory management and message passing
- Run all non-essential services of the OS in **user mode**
  - Communication between them with message passing

- Advantages
  - High stability
    - A bug in driver cannot take the entire system down
  - High flexibility
    - System extension do not need to rebuild the kernel

- Disadvantages
  - High overhead from message passing

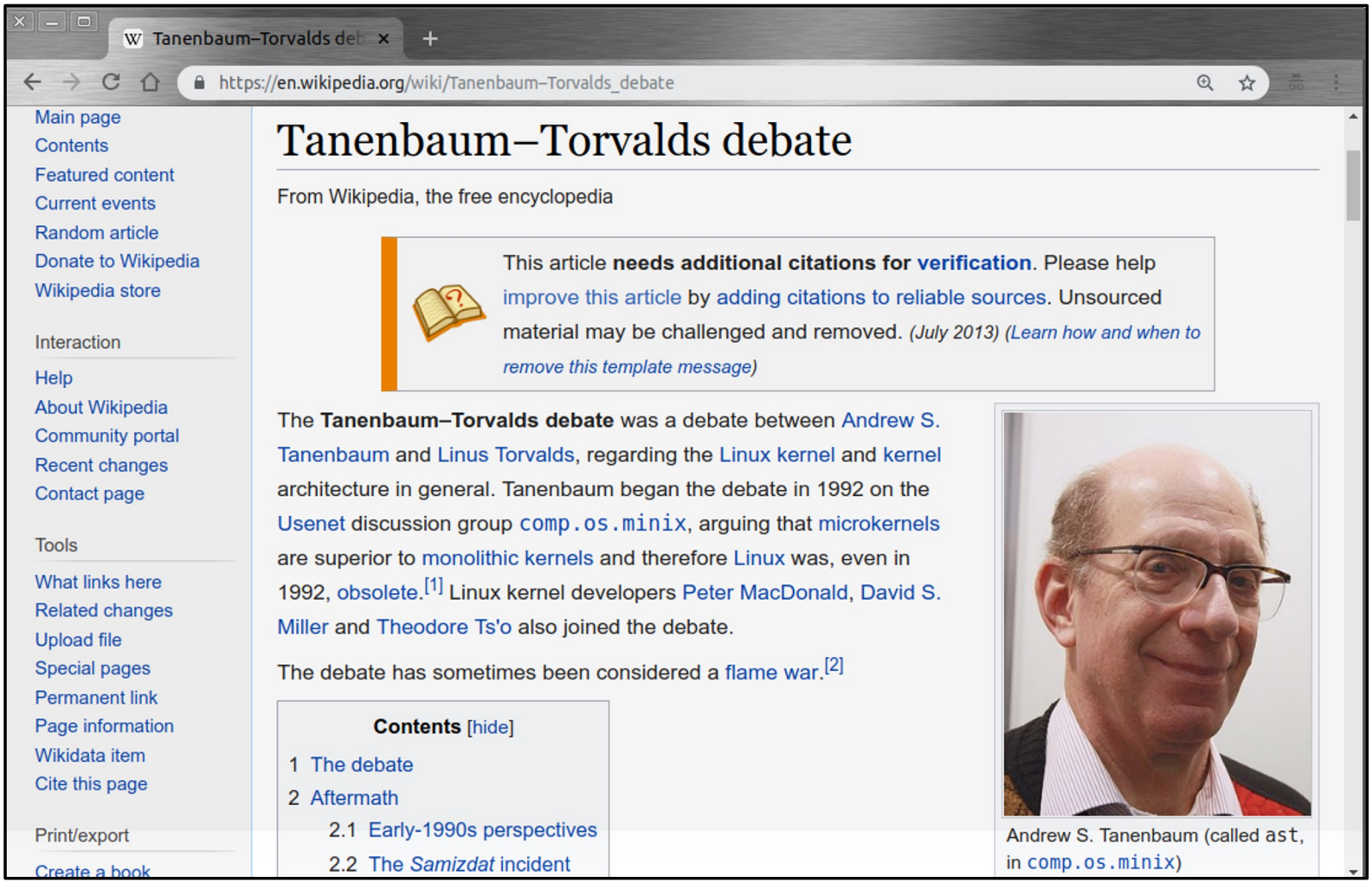
# Monolithic Systems vs Microkernels




Source:
wikipedia

# Hybrid Systems

- Many modern operating systems do not strictly adhere to the monolithic / microkernel models
  - **NT kernel** in post-2000 Windows, including phones and Xbox
  - **XNU kernel** in macOS, iOS, watchOS


Tanenbaum–Torvalds deb
https://en.wikipedia.org/wiki/Tanenbaum–Torvalds_debate
Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store
Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact page
Tools
What links here
Related changes
Upload file
Special pages
Permanent link
Page information
Wikidata item
Cite this page
Print/export
Create a book
Tanenbaum–Torvalds debate
From Wikipedia, the free encyclopedia
This article needs additional citations for verification. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed. (July 2013) (Learn how and when to remove this template message)
The Tanenbaum–Torvalds debate was a debate between Andrew S. Tanenbaum and Linus Torvalds, regarding the Linux kernel and kernel architecture in general. Tanenbaum began the debate in 1992 on the Usenet discussion group comp.os.minix, arguing that microkernels are superior to monolithic kernels and therefore Linux was, even in 1992, obsolete.[1] Linux kernel developers Peter MacDonald, David S. Miller and Theodore Ts'o also joined the debate.
The debate has sometimes been considered a flame war.[2]
Contents [hide]
1 The debate
2 Aftermath
2.1 Early-1990s perspectives
2.2 The Samizdat incident
Andrew S. Tanenbaum (called ast, in comp.os.minix)